

EMKS 技術白皮書

Enterprise Multi-role Knowledge System

整合 LangGraph Agent 與 RBAC-aware RAG 之企業文件系統技術文獻

版本：2026-05-15 · 作者：蔡易霖

EMKS 解決什麼問題

當企業累積大量內部文件，員工花在「找對資料」上的時間遠超直覺；而直接導入通用 AI 工具，又會撞上資料權限失控與內容失準之合規問題。

問題情境

中型 B2B 企業（典型規模 50-200 人、具備 3-5 個明確部門）之內部知識（SOP、產品手冊、政策、技術筆記）通常分散於 wiki、雲端硬碟、Slack 釘訊息、各部門共用磁碟。員工每天為「找對的文件」付出可觀時間，且資訊查得到與否成為新員工 ramp-up 速度之主要瓶頸之一。

直接導入通用 AI 工具（如 ChatGPT Enterprise、Notion AI）看似為快速解法，但於企業場景將撞上三個結構性問題：

- **AI 不認識企業既有之權限系統** — 員工問什麼答什麼，無法分辨「該看」與「不該看」之內容，違反 RBAC 等同合規事件。
- **AI 對企業特定內容會編造** — 對「我們公司之採購流程」此類問題，通用 LLM 將產出「看似合理但實為業界通用範本」之回答，具高度誤導性。
- **AI 介面將工程細節推給使用者** — 要求使用者自行判斷「該走 RAG 或 Agent、該開哪個 tab」等同將後端架構抽象洩漏至使用者介面。

EMKS 之解決方案

員工只看得到他該看的

文件入庫時，部門與敏感等級即綁進 AI 的索引；員工查詢時，他無權限的內容根本不會被搜到 — 不是搜出來再篩掉，而是壓根搜不出來。
(技術細節詳見 §3)

一個對話框打天下，使用者不必選 RAG 還是 Agent

不分 tab、不選模式，所有問題打進同一個對話框。系統自己判斷要查知識庫、列近期文件、還是跨步驟組合查詢。(技術細節詳見 §4)

斷網不丟答案、關頁不燒錢

網路斷、頁面關、伺服器重啟，使用者已看到的回答不會消失；使用者離開後，後端也不會繼續消耗 AI 服務費用。(技術細節詳見 §5)

系統功能大綱

以下列出系統主要功能；每項末尾標註對應之技術章節，供有興趣之讀者深入：

AI 智慧問答 — 員工於單一對話框提問，由系統自動判斷查詢知識庫、列出近期文件、或進行跨步驟組合查詢。 §4 · §6

權限隔離與資料合規 — 文件依部門與敏感等級寫入 AI 索引，員工無權限之內容於搜尋階段即不可見；同時涵蓋 AI 對話與直接 API 存取兩條路徑。 §3

文件審核 workflow — 新文件與新版本須主管審核通過後始進入知識庫；駁回需附原因，被駁回之版本不影響線上版本。 §7.2.3

多角色多部門組織管理 — 四階角色（admin / manager / employee / viewer）+ 部門隔離 + 細粒度權限碼，支援「能審核但不能跨部門看資料」之中階主管角色。 §3.6 · §7.2.1

文件版本控制與收藏 — 新版本不覆蓋舊版（INSERT 機制）；軟刪除支援復原；SHA-256 檔重複上傳；員工可收藏常用文件、樹狀資料夾分類。 §7.2.2

企業級認證 — JWT 自動續發、帳號連續失敗 5 次鎖定、密碼重設與啟用信流程、refresh token 可撤銷。 §7.1.1 · §7.1.2

登入安全與審計 — 完整登入歷史追蹤；防 email enumeration；雙維度 rate limit（帳號級防鎖死、IP 級防分散撞庫）。 §7.1.3

雲端 / 本地部署 — Docker compose 一鍵啟動；stateless 設計適合雲端 PaaS；環境變數注入支援多環境配置；含冷啟動 health check 與前端 polling。 §8

本頁為業務面摘要；後續章節為技術實作之完整論述，包含設計決策、反方案分析與已知限制。

摘要

EMKS (Enterprise Multi-role Knowledge System) 為一整合 RBAC、文件審核流程與 AI 問答能力之企業知識管理系統。其技術定位非生產環境部署，而為一示範性實作，目的在於展示如何將 LLM 整合至帶有業務約束（多角色權限、審核流程、跨部門隔離）之既有系統。此議題對應 B2B SaaS 將 AI 導入既有產品線時所面對之工程挑戰。

系統設計圍繞三項核心決策：

1. **RBAC-aware RAG 三層權限驗證機制**：於文件向量化階段將權限資訊寫入 ChromaDB metadata，使搜尋階段能透過 **where** 子句進行 pre-filter；配合 LangGraph 動態 tool binding 與 tool 內部二次驗證，形成 defense in depth 之結構。
2. **LangGraph 統一入口架構**：摒棄「RAG tab / Agent tab」分流介面，所有對話經由單一 **/ai/agent** SSE endpoint 進入，由 Agent 透過 LLM 原生 tool calling 進行 dispatch 決策。
3. **SSE 串流之三層防護機制**：在跨 async/sync 邊界之串流場景中，分別針對斷線偵測、Service 層 SessionLocal 隔離、與 ContextVar 複製時機三個故障點建立對應之防護。

本文針對上述三項核心決策與輔助子系統（認證、文件管理、部署運維）提供完整之技術說明、反方案對照分析、以及已知限制。讀者預設為熟悉 Python 後端、Vue 前端、與 LLM 應用層基礎之工程師。

關鍵字：RAG · LangGraph · Defense in depth · SSE · FastAPI · ChromaDB · Multi-tenant RBAC

目錄

1. 引言
2. 系統概觀
3. RBAC-aware RAG：三層權限驗證機制
4. 統一入口：LangGraph Agent 架構決策
5. SSE 串流：三層防護機制
6. RAG 核心機制
7. 輔助模組
8. 部署與運維
9. 工程品質
10. 已知限制與技術債
11. 討論
12. 結論
13. 附錄

1. 引言

1.1 問題背景

當 B2B SaaS 產品將 AI 能力導入既有系統時，所面對之工程挑戰與獨立開發新 chatbot 產品存在本質差異：

- 核心問題不在於「LLM 能否回答」，而在於「LLM 之回應是否僅涵蓋使用者具備權限之內容」。資料權限為企業客戶之強制需求，違反此項即構成合規事件。
- 核心問題不在於「LLM 之能力上限」，而在於「LLM 失誤時系統之防護能力」。Prompt injection、幻覺、tool call 誤判等皆為實際存在之風險。
- 核心問題不在於「介面是否導入 AI」，而在於「AI 是否應外顯為介面元素」。要求使用者選擇「該問題應送往 RAG 或 Agent」此類設計，等同將內部架構抽象洩漏至使用者介面。

EMKS 將上述挑戰作為明確之設計約束處理，於系統設計初期即納入，而非於 AI 元件整合後另行補強。

1.2 系統定位

EMKS 為技術示範性系統，非生產級實作。具體範圍界定如 Table 1.1。

Table 1.1 EMKS 系統範圍界定。

範圍項目	是否包含於本系統
完整 RBAC 模型與多角色組織結構	是
文件審核流程與版本控制	是
AI 問答端到端流程（RAG、Agent、Streaming）	是
雲端部署與多環境配置	是
RAG 品質評估資料集（precision@k / recall@k）	否，列為已知技術債
生產級可觀測性（trace / metric / alert）	否，僅實作結構化 logging
多租戶完整支援	模型層支援，未經實際驗證
真實使用者驗證迭代	否，採同行評審替代

1.3 核心設計原則

系統設計自三項原則展開：

原則 1：權限為資料屬性而非程式邏輯

RBAC 不應僅止於「角色對應頁面」之 gating 設計。資料層級之權限隔離應於資料儲存階段固化於資料本身，使搜尋階段能直接 pre-filter，而非於搜尋結果產生後再行篩選。此原則為第三層權限驗證之物理基礎。

原則 2：統一入口優於分流介面

令使用者選擇技術架構為錯誤之抽象設計。Agent 形態之系統應由 Agent 自身完成分流決策，介面則收斂為單一對話框。

原則 3：Defense in depth

AI 系統對 prompt injection、tool call 誤判等攻擊面具有特殊脆弱性。單一防線受破壞即構成資料外洩，因此採用多層 redundant 設計，使任一層失效不致導致整體系統失守。

2. 系統概觀

2.1 技術棧

本系統採用之技術棧列示於 Table 2.1。

Table 2.1 EMKS 系統技術棧。

層級	選用技術
前端	Vue 3 + Vite 7 + Pinia + axios
後端	FastAPI + Uvicorn
ORM	SQLAlchemy 2 (async-ready，本系統採 sync 模式)
關聯式資料庫	MySQL 8
向量資料庫	ChromaDB (embedded sqlite 模式)
LLM 框架	LangGraph + LlamaIndex (用於 query condensation)
LLM 服務	OpenAI API (gpt-4o-mini + text-embedding-3-small)
部署平台	Render (後端) + Vercel (前端) + Docker Compose (本地)

2.2 整體架構

系統整體架構如 Figure 2.1 所示，分為前端、後端、儲存層與外部服務四個區塊。

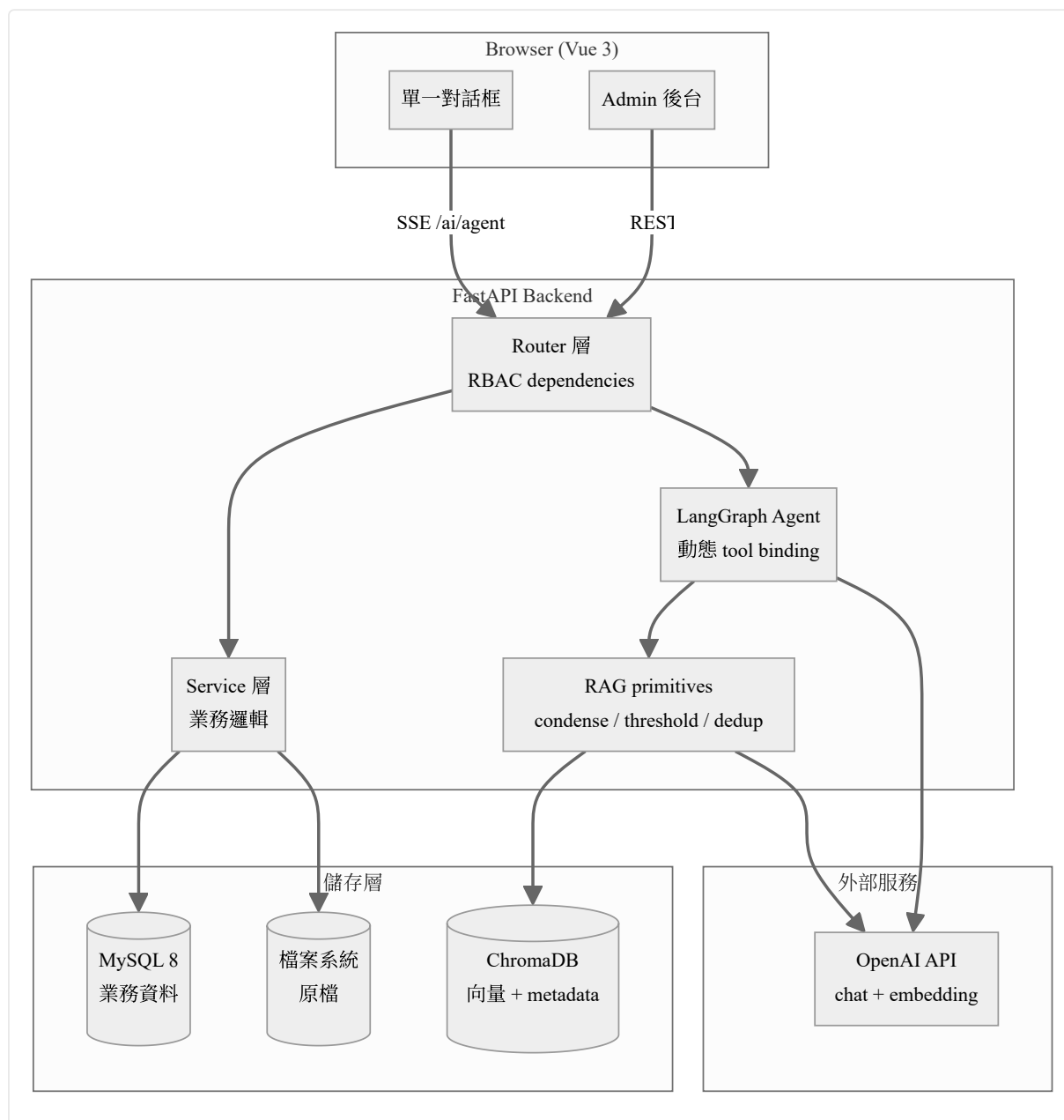


Figure 2.1 EMKS 系統整體架構。前端透過 SSE 進入後端統一 AI 入口，後端 Router 層套用 RBAC dependencies 後分派至 Service、Agent 或 RAG 子系統，再分別與 MySQL、ChromaDB、檔案系統與 OpenAI 服務互動。

2.3 模組劃分

系統依功能職責劃分為五個模組，於本文中之展開順序如 Table 2.2。本文採功能重要性而非字母順序進行展開。

Table 2.2 模組劃分與本文章節對應。

模組	內容	對應章節
D. AI 核心	向量化、RAG、Agent、統一入口、SSE	§3, §4, §5, §6
B. 權限系統	RBAC 模型、三層驗證、前後端雙守衛	§3（併入三層驗證討論）
A. 認證	JWT auto-refresh、帳號生命週期、登入安全	§7.1
C. 文件管理	組織 CRUD、資料夾與版本、審核流程	§7.2
E. 部署運維	Stateless demo、多環境配置	§8

3. RBAC-aware RAG : 三層權限驗證機制

3.1 問題：AI 系統之權限挑戰

傳統 web 應用之 RBAC 設計通常聚焦於兩個層面：(1) 角色對應頁面之 gating、(2) 角色對應 API endpoint 之准入控制。此設計於純 CRUD 系統足以滿足需求，但於整合 AI 問答後產生新的威脅模型：

- **語意搜尋繞過 endpoint 過濾**：使用者查詢觸發 ChromaDB top-K 向量檢索。若 ChromaDB 返回完整結果集後由應用層過濾，「不應被存取之內容」已離開向量資料庫之邊界。
- **LLM tool calling 介面構成攻擊面**：Tool function 之 signature 即為 LLM 所見之 schema。若 `user_id` 設計為 function parameter，prompt injection 可能誘導 LLM 填入其他 user_id。
- **單點防線易受繞過**：攻擊者繞開 AI 路徑、直接呼叫 REST endpoint（如 `/documents/{id}/download`）需有對等之保護機制。

三層權限驗證機制針對上述三類問題分別建立對應之保護層。

3.2 三層權限驗證機制

整體機制之結構如 Listing 3.1 所示。

Listing 3.1 三層權限驗證機制之結構與職責劃分。

```
第一層（功能級結構過濾）
  位置：_build_agent_app(is_admin)
  機制：根據角色動態組裝 tool list，再呼叫 llm.bind_tools(tools)
  效果：non-admin 之 LLM 不會於 schema 中看見 admin tool

第二層（功能級條件驗證）
  位置：每個 tool function 內部
  機制：if not ctx['is_admin']: return "您沒有權限"
  防護對象：refactor 缺失、cache 污染、新增 tool 時遺漏 bind 邏輯
           等程式碼層級之 regression

第三層（資料級 pre-filter）
  位置：ChromaDB query 階段
  機制：where = build_permission_filter(is_admin, dept_id)
         chroma.query(embedding, where=where)
  效果：權限不符之向量不會進入應用層
```

3.3 物理基礎：Metadata 階段固化權限

第三層之有效性依賴於向量化 pipeline（D1，詳見 §6.1）於寫入時將權限資訊固化至 ChromaDB metadata。寫入結構如 Listing 3.2。

Listing 3.2 ChromaDB chunk 寫入時之 metadata 結構。

```
chunk_metadata = {
    "document_id": doc.id,
    "chunk_index": i,
    "permission_level": doc.permission_level,    # public / department
    "department_id": doc.department_id,         # 跨部門隔離鍵
    "document_title": doc.title,
}
collection.add(
    ids=[chunk_id],
    embeddings=[embedding],
    documents=[chunk_text],
    metadatas=[chunk_metadata],
)
```

`build_permission_filter` 函式依使用者角色與部門資訊產生對應之 ChromaDB `where` 子句，三種情境之輸出如 Table 3.1。

Table 3.1 不同角色情境下之 ChromaDB `where` 子句結構。

角色情境	<code>where</code> 子句
admin	<code>None</code> （無過濾）
non-admin， 具部門歸屬	<code>{"\$or": [{"permission_level": "public"}, {"department_id": dept_id}]}</code>
non-admin， 無部門歸屬	<code>{"permission_level": "public"}</code>

此設計之結構性意義為：權限驗證發生於資料檢索之前，而非檢索之後。若 D1 階段未寫入對應 metadata，第三層僅能執行 post-filter，此時不符權限之內容已離開向量資料庫之保護邊界。

3.4 反方案分析

若拆除任一層驗證機制，對應之失敗模式如 Table 3.2。

Table 3.2 移除任一層驗證機制後之失敗模式分析。

移除項目	失敗模式
第一層（改為靜態 bind 全部 tools，內部判斷 role 拒絕）	LLM schema 暴露 admin tool 之存在；prompt injection 攻擊面擴大；schema 本身即構成資訊洩漏
第二層（僅依賴 bind 層）	第一層因 refactor、cache 污染或新增 tool 時之遺漏所產生之 regression 將無對等防護
第三層（tool 通過後直接 query）	ChromaDB 進行全庫向量檢索；跨部門資料於語意相關性高時將出現於檢索結果
將 <code>is_admin</code> 改為 tool function parameter	LLM 於 tool_call 時可填入此欄位；prompt injection 可誘導其填入 <code>True</code>
採用 <code>threading.local</code> 傳遞 user context	LangGraph ToolNode 之 worker thread 無法存取，導致權限檢查失效

正確之 user context 傳遞方式為 Python `contextvars`。此選擇之依據為：(1) 對 LLM 透明，不會出現於 tool schema；(2) 透過 framework 之 `copy_context()` 機制自動跨 thread 傳遞。

3.5 設計缺陷之識別與修補

初版三層驗證機制完整覆蓋 AI 對話路徑，但檢視 REST surface 後識別出語意缺口：跨部門使用者於 AI 問答中無法檢索受限文件（被第三層 metadata filter 擋下），然直接呼叫文件下載類 REST endpoint 仍可成功取得內容。

根因分析顯示，`build_permission_filter` 之設計屬性為 view filter（搜尋結果過濾），而非 access gate（資源存取守門）。三層機制覆蓋了 AI 對話 surface，但繞過 AI 路徑、直接呼叫 REST endpoint 之攻擊路徑未受同等保護。

修補方式為將同等規則下沉至 resource 層，新增 `can_access_document` 與 `build_document_access_filter` 兩個 helper，其語意與 `build_permission_filter` 完全對齊（包含 `ai:admin_tools` 之 bypass 邏輯），並套用至 list、get、download、upload_new_version、delete、restore、versions 共 7 個 endpoint。

第二輪迭代中進一步識別出：上述修補僅驗證「使用者對該文件之存取權」，未驗證「該角色對該操作之執行權」。Viewer 角色於修補後仍可呼叫 POST `/documents` 上傳檔案、DELETE 其可見之 public 檔案。修補方式為擴充 PERMISSIONS seed，補上 `document:create / edit / delete` 等權限碼，並將 4 個 write endpoint 改為 `Depends(require_permission("document: ..."))` 形式。

由上述兩輪迭代可歸納兩項設計準則：(1) Pre-filter 與 access gate 為不同語意，RBAC 規則應於每條對外 surface 各自實作一次，此重複實作為 defense in depth 之定義性需求而非冗餘；(2) Resource-level（特定資料之存取權）與 role-level（角色之操作權）為兩個正交維度，完整 RBAC 機制應於兩個維度均建立驗證。

3.6 Permission code 設計

本系統採用 `resource:action` 雙層格式之權限碼（例：`user:create`、`document:edit`、`ai:admin_tools`）。扁平結構利於查詢，未來新增權限碼不涉及 schema 變更。

本設計遵循之判斷準則為：權限碼之本質為 data 而非 code。判斷依據為「該項目之變更是否需要 code release」；不需要者屬 data，應儲存於資料庫；需要者屬 code，可考慮 enum 形式。將運行期可擴充之 data 鎖入 code 層將限制多租戶 SaaS 之客戶自助配置能力與 incident 之分鐘級熱修復能力。

AI 管理權限與後台管理權限採分離之權限碼設計（`ai:admin_tools` 與 `user:*`），支援兩個正交維度之組合，如 Table 3.3。

Table 3.3 後台 admin 與 AI admin 之正交組合範例。

角色	後台 admin	AI admin
IT 主管	是	是
資料總監	否	是
系統管理員	是	否
一般員工	否	否

四種組合均為實際存在之角色配置需求，故權限模型需於初期即支援此正交性。

4. 統一入口：LangGraph Agent 架構決策

4.1 問題：RAG/Agent 介面分流之缺陷

系統初版設計於 UI 層提供「RAG 知識庫問答」與「AI Agent」兩個分離之 tab。使用者需自行選擇問題對應之路徑。實際使用過程中識別出三類問題：

問題 1：使用者認知負擔

RAG 與 Agent 為內部技術概念。一般使用者無法判斷其問題所屬之類別。錯誤分類（如將知識庫問題提交至 Agent tab）需經過退回與重問之流程。要求使用者選擇技術架構之設計，等同將工程內部抽象洩漏至使用者介面。

問題 2：Agent tab 之幻覺風險

未強制走 knowledge base 之條件下，Agent 將以 base model 之能力產生回應。針對企業特定主題（如報銷流程、SOP），base model 可能產出語法合理但內容錯誤之 generic SOP 模板。對企業知識工具而言，此屬高風險錯誤類型。

問題 3：架構抽象與介面抽象之不對應

於 LangGraph 框架下，`search_knowledge_base` 本即為 Agent 可呼叫之 tool 之一。將其於 UI 層分離為獨立 tab，等於令使用者察覺工程內部之 tool 切分。

4.2 統一入口設計

合併後之介面收斂為單一對話框，後端對應單一 `/ai/agent` SSE endpoint。三種典型查詢類型之資料流如 Figure 4.1。

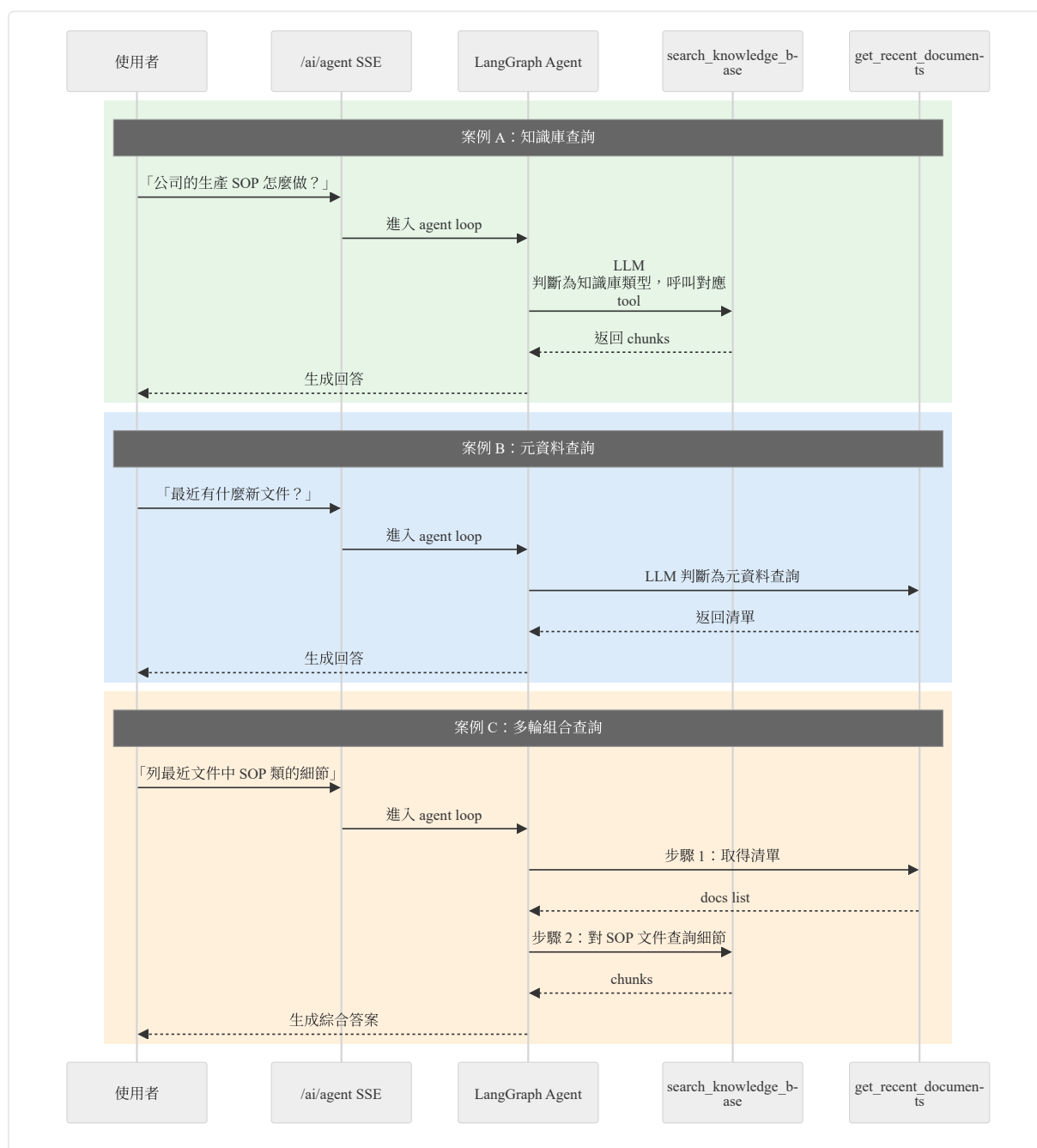


Figure 4.1 統一入口架構下三種典型查詢之資料流。所有查詢進入單一 SSE endpoint，由 LangGraph Agent 內部依問題性質 dispatch 至對應 tool。多輪查詢透過 agent loop 之 state 累積實現。

此設計之結構性意義為：「統一」發生於使用者介面層，而非後端架構層。後端因 agent decision 與 state 累積需求而複雜度上升，但複雜度對使用者透明。此屬產品複雜度由前端向後端轉移之決策。

4.3 LangGraph Agent 之內部機制

Agent 內部運行於一個有限狀態機之上，其狀態轉移如 Listing 4.1。

Listing 4.1 LangGraph Agent 之狀態轉移結構。



本系統實作之四個內建 tool 列於 Table 4.1。

Table 4.1 LangGraph Agent 內建之四個 tool 及其 RBAC 限制。

Tool	用途	RBAC 限制
search_knowledge_base	RAG 主搜尋	所有角色
get_recent_documents	近期文件清單	所有角色
get_pending_reviews	使用者自身之待審文件	所有角色（限自身範圍）
get_popular_questions	熱門查詢統計	admin 限定

動態 tool binding 機制（即三層驗證之第一層）：`_build_agent_app(is_admin)` 依角色組裝 tool list 後 呼 叫 `llm.bind_tools(tools)`。Non-admin 角色之 LLM schema 中 不 會 出 現 `get_popular_questions`。

4.4 正確性保證：hard layer 與 soft layer 之分層

統一入口架構下，「應走 knowledge base」之正確性透過兩層機制保證：

Hard layer（主要保證）：GPT-4o tool calling bias 與 system prompt

- System prompt 明確要求「優先使用 knowledge base；不確定時應明示」
- GPT-4o 配合 well-defined tool schema 時，多數情境下傾向呼叫對應 tool 而非直接生成回答
- 大部分企業主題查詢將透過 tool 處理而非進入 base model 推論

Soft layer（次要偵測）：Base model 之統計性偏差

- GPT-4o 未經本公司資料訓練。即使未呼叫 tool 直接生成，產出內容於統計上將泛化、添加 hedge 語句、傾向於 generic SOP 模板形式
- 使用者於閱讀時可能識別出「語氣與本公司不符」，從而觸發對話迭代修正

需指出之限制：上述機制非結構性保證。「base model 未經 X 訓練」並不蘊含「base model 不會產出類似 X 之內容」；對於各組織間具高度相似性之主題（如報銷流程、OKR），soft layer 之偵測能力顯著下降。因此 hard layer 為主要防線，soft layer 為輔助偵測層。

4.5 反方案分析

各反方案之失敗模式如 Table 4.2。

Table 4.2 統一入口架構之反方案失敗模式分析。

反方案	失敗模式
保留 RAG/Agent tab 分流	同時觸發三項問題：認知負擔、Agent 幻覺、架構抽象洩漏
僅保留 Agent tab	未強制走 KB 導致幻覺風險擴大
僅保留 RAG tab	失去多 tool 能力（無法處理近期文件、待審、熱門等元資料查詢）
採自寫 router 進行問題分類（regex 或 classifier）	分類規則覆蓋不全（多步問句之分類困難）；規則表構成獨立之維護負擔；失去 LangGraph 之 multi-step state 累積能力
System prompt 完全放手	Base model 於通用主題產出仿真但內容錯誤之回應，soft layer 之偵測能力不足
System prompt 強制走 tool	簡單 chitchat 亦觸發 tool 呼叫，增加延遲與成本，UX 退化

自寫 router 與 LLM tool calling 之關鍵差異不在「簡單問題之分流能力」，而在「多步查詢之 state 累積能力」。以「列最近文件中，SOP 類的細節是什麼？」為例：

- 自寫 router 需先解析「最近文件」分派至 endpoint A 取得清單，再解析「SOP 細節」分派至 endpoint B，但兩個 endpoint 間無 cross-step 上下文，state 維護需於應用層額外實作。
- LLM tool calling 配合 LangGraph：tool 回傳後 state 自動累積，LLM 於下一輪推論時可存取清單並決定後續操作，自然支援 multi-step。

由此可見，tool calling 之核心價值不在「dispatch 功能」，而在「agent loop 之 state 累積機制使多步查詢之表達成為可能」。

4.6 Persona 適用範圍

本架構決策之適用範圍依使用者群體特性而異，如 Table 4.3。

Table 4.3 不同 persona 下之適用性分析。

使用者群體	適用性調整
純工程使用者（DevOps、SRE）	分 tab 設計反而符合需求 — 使用者明確知道查詢類型，自動 dispatch 反而干擾控制感
高風險領域（法律、醫療）	可能需強制走 RAG（避免 agent 自由發揮），架構退化為「僅 RAG tab」形式，但失去多 tool 能力
多模態應用（圖像、影音）	統一入口仍適用，但 tool 集需大幅擴充

本系統之設計 pin 於「80 人規模、台灣 B2B SaaS、多數非工程使用者」之假設下。於此假設外之 persona，設計決策需重新評估。

5. SSE 串流：三層防護機制

5.1 問題：LLM 串流跨 async/sync 邊界之挑戰

LLM 生成長回答之延遲約為 20 秒。UX 需求如下：

1. **逐字串流顯示**：一次性顯示完整回答將造成 20 秒之空白等待。
2. **使用者中斷之 backend 感知**：未感知斷線將導致 OpenAI stream 持續運行，token 計費與 session 資源未釋放。
3. **partial answer 保留**：未保留將導致使用者重新提問又花費 20 秒生成。

實作上存在以下技術約束：

- EventSource API 不支援 Authorization header，JWT 無法透過標準 SSE 機制傳遞，須改用 `fetch` + `ReadableStream` 自行實作。
- FastAPI `StreamingResponse` 無內建之「對方斷線」通知機制，generator 預設無法得知 client 已離線。
- LangGraph 之 `stream()` 為 sync API。於 async router 內使用需透過 `iterate_in_threadpool` 推至 worker thread，引入跨 thread 之技術約束。
- SQLAlchemy session 不具備 thread safety，跨 thread 共用 session 物件將導致 race condition。
- Python `contextvars` 跨 thread 之傳遞依賴 framework 呼叫 `copy_context()`，set 之時機若不正確將導致複製至錯誤之 thread-local copy。

5.2 三層防護機制

三層防護之資料流如 Figure 5.1 。

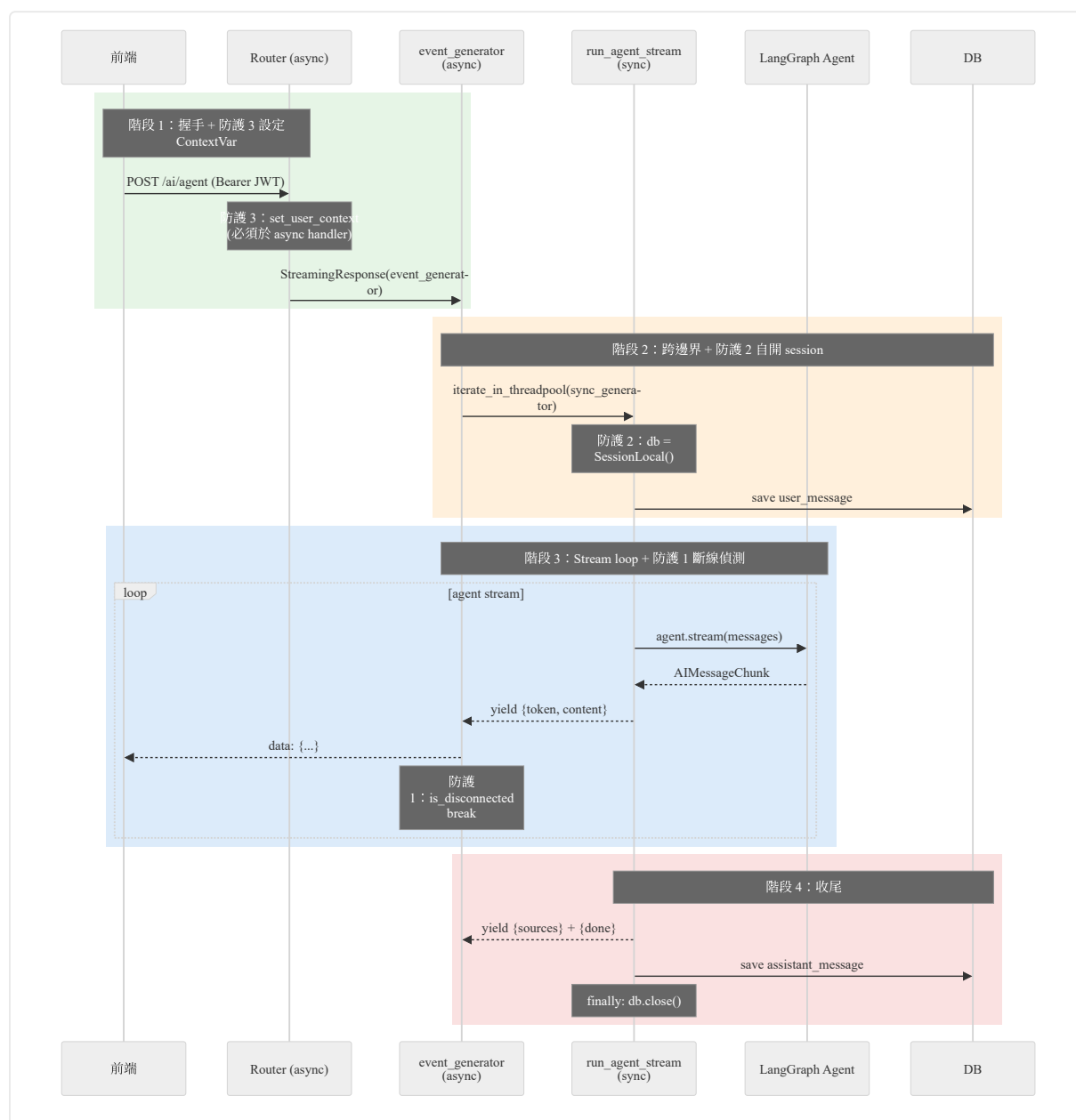


Figure 5.1 SSE 三層防護機制之資料流。階段 1 於 async handler 設定 ContextVar (防護 3)；階段 2 於 sync generator 內自開 SessionLocal (防護 2)；階段 3 於 async wrapper 進行 is_disconnected polling (防護 1)。

三層防護之機制與作用範圍如 Table 5.1 。

Table 5.1 三層防護機制之職責與必要性層級。

防護層	機制	解決問題	必要性層級
1. is_disconnected polling 與 finally 關閉 stream	Async wrapper 於每次取 chunk 前執行 <code>await</code> <code>request.is_disconnected()</code> , 斷線即 <code>break</code> , <code>finally</code> 區塊執行 <code>sync_generator.close()</code>	使用者中斷時關閉 OpenAI stream、保留 partial answer	適用於任何 SSE 實作
2. Service 層自開 SessionLocal	Generator 內以 <code>db =</code> <code>SessionLocal()</code> 建立獨立 session , <code>finally</code> 區塊執行 <code>db.close()</code>	跨 thread 操作 SQLAlchemy session 引發之 race condition 與 connection pool 耗盡	sync stream + threadpool 路徑之硬性約束
3. ContextVar 於 async handler 設定	<code>set_user_context()</code> 必須於 router async handler 之入口呼叫 (即 <code>iterate_in_threadpool</code> 之前)	Tool 內 <code>get_user_context()</code> 取得 <code>None</code> 或 <code>KeyError</code>	sync stream + threadpool 路徑之硬性約束

5.3 分層機制之識別過程

本機制非一次性規劃之結果，而為實作過程中逐步識別。各層之觸發症狀、技術根因與實作方式如下：

防護 1：is_disconnected 與 finally 關閉 stream

觸發症狀為使用者關閉頁面後 OpenAI stream 持續運行（持續計費、session 未釋放）。實作為於 router 層宣告 async `event_generator()` wrapper，於迴圈內進行 `is_disconnected()` polling 並於 `finally` 區塊關閉下游 sync generator。此層之適用範圍最廣，不依賴 LangGraph 或 threadpool 之特性。

防護 2：Service 層自開 SessionLocal

觸發症狀為斷線時 partial answer 寫入失敗、偶發之 SQLAlchemy session thread-safety 錯誤、connection pool 異常增長。根因為 sync generator 運行於 worker thread、async router 運行於 event loop，借用 router 注入之 db session 構成跨 thread 共用。實作為於 generator 內自建 `SessionLocal()`。此層為 sync stream + threadpool 路徑之硬性約束。

防護 3：ContextVar 於 async handler 設定

觸發症狀為 tool 內 `get_user_context()` 偶發返回 `None`。根因為 `iterate_in_threadpool` 進入 worker thread 之瞬間將複製當前 context；若 ContextVar 於 generator 內才設定，進 thread 之瞬間 context 尚未帶有 user 資訊，後續之 set 操作將作用於 thread-local copy。實作為將 `set_user_context()` 移至 router async handler 之入口。此層亦為跨 thread 邊界之硬性約束，但與防護 2 為不同 vector（前者解 context 傳播、後者解 ORM thread-safety）。

三層雖各自有獨立之觸發症狀與技術根因，但作用範圍存在強耦合：若無防護 1，generator 將自然結束釋放 session，防護 2 之必要性無法穩定觸發；若無防護 2，偶發之 session 錯誤將掩蓋防護 3 之 ContextVar 症狀。三層累積後方形成完整之「跨 async/sync 邊界 SSE」實作。

由此可見，分層防護機制之 narrative 為實作過程中逐步顯現之結果，而非規劃階段之上層設計。對外描述採用 defense in depth 等 pattern 詞彙具描述精確性，但須區別「結構之適用性」與「結構之來源」。

5.4 反方案分析

各反方案之失敗模式如 Table 5.2。

Table 5.2 SSE 三層防護之反方案失敗模式。

反方案	失敗模式
採用 WebSocket 取代 SSE	JWT 認證需自定 protocol；雙向通道於本場景未被使用；反向代理之 WebSocket 配置複雜度高於 SSE
採用 EventSource 取代 fetch + ReadableStream	JWT 須以 query string 傳遞，造成 URL log、referer、browser history 洩漏；改採 cookie 認證將牽動整個 auth 流程重構
於純 async generator 內直接迭代 sync <code>stream()</code>	<code>for chunk in sync_stream</code> 於 async function 中將 block event loop，導致 FastAPI 整體阻塞
借用 router 注入之 db session	跨 thread 共用 SQLAlchemy session 引發 race condition 與 connection pool 異常
不偵測斷線	OpenAI stream 持續運行至 20 秒結束，token 浪費與 worker thread 未釋放

另一條改寫路徑為換用 LangGraph 之 async-native `astream()`。此路徑將消除 `iterate_in_threadpool` 之需求，防護 2 與防護 3 將不再為硬性約束。本系統未採此路徑之原因為：sync 路徑經三輪修補後已穩定，改寫至 `astream` 需重構整段 generator 與 ContextVar 處理邏輯，於示範性系統之範圍內 cost 高於 benefit。此項列為已知技術債。

6. RAG 核心機制

6.1 向量化 Pipeline

文件審核通過後進入向量化 pipeline，採以下四項設計：

設計 1：Chunk 重疊滑動窗口（500 字 + 50 字 overlap）

滑動窗口起點每次前移 $size - overlap = 450$ 字，相鄰窗口間有 50 字之重疊。

- 反方案 `overlap=0`：若關鍵語句（如「明天會議改到三樓」）切於兩個 chunk 之邊界，兩個 chunk 之 embedding 將分別缺失上下文，使「會議在哪？」之查詢無法命中關鍵 chunk。
- 反方案 `overlap=200`：embedding 計算重複 4 倍，context window 利用率下降。
- 50 字之選擇對應一個短句之長度，邊界資訊於兩側皆得以保留。

設計 2：Batch embedding

整份文件之所有 chunks 透過 `get_embeddings_batch(chunks)` 一次性提交至 OpenAI API。

- 反方案逐 chunk 呼叫：跨網路呼叫之固定 overhead 為 200-500 ms，30 chunks 累積延遲為 6-15 秒。
- Batch 呼叫之延遲約為 500 ms，總延遲下降 12-30 倍。
- 此屬與 SQL N+1 query 同類之反 pattern。

設計 3：Metadata 隨 chunk 儲存（denormalize for read）

寫入 ChromaDB 時將 `permission_level`、`department_id` 等權限資訊作為 metadata 一併寫入。

- 反方案：搜尋結果產生後回 MySQL 查詢權限，引入額外之 DB round-trip 與 N+1 query。
- 採用 denormalize：權限資訊與 chunk 共存於 ChromaDB，向量比對階段直接過濾。
- 取捨：寫入路徑延長（權限變更需 reindex），讀取路徑加速。讀寫比於本場景約為 100:1 以上，成本側重於寫端之選擇合理。

設計 4：Defensive delete（寫入前先刪除）

Chunk ID 採固定格式 `doc{doc_id}_chunk{i}`。每次向量化前先刪除前綴匹配之所有 chunks 再寫入新版本。

- 反方案直接 add：若新版本切出 8 chunks 而舊版本為 10 chunks，`chunk_8`、`chunk_9` 將殘留於資料庫，導致查詢命中舊版內容。
- 採用先刪後寫：操作具備冪等性，重複執行 N 次最終狀態相同。

6.2 RAG 搜尋四機制

D2 之核心議題並非「文字搜尋」之精度，而為「LLM 取得之 context 品質對最終回答品質之影響」。針對四個失效模式各設計一項機制。

機制 1：Condense — 多輪追問之語意還原

多輪對話中，使用者之追問常以省略形式出現。例：

Listing 6.1 多輪追問之指代失焦範例。

```
User: 公司的生產 SOP 有哪些？
AI: 有 SOP-A、SOP-B、SOP-C
User: B 是什麼？ ← 5 字之短查詢，embedding 結果偏向字母 B 之語意
```

採用 LlamaIndex `condense_question` 將追問與 chat history 提交至 LLM 改寫為獨立查詢（如「公司生產 SOP 中 B 是什麼？」），改寫後之查詢進入 embedding 與 ChromaDB 搜尋。

此機制僅於 chat history 非空時觸發。首輪查詢本身為獨立形式，多餘之 condense 步驟將增加約 300 ms 延遲。

機制 2：Threshold filter (門檻 0.35)

ChromaDB 採 cosine similarity 計算 top-K 結果，但 top-K 之返回不保證所有結果皆為高相關性。當主題冷門時，剩餘 slot 將被低分結果（cosine 約 0.1）填滿。本系統採 `relevance < 0.35` 之過濾門檻。

反方案（全部餵入 LLM）之失敗模式為：低分 chunks 作為 distractor，LLM 對所有 context 進行 conditioning，verbose 之低相關 chunk 反而較精簡之高相關 chunk 更容易被引用。此屬 accuracy 層面之退化，非 UX 層面之問題。

機制 3：Dedup by document_id

LangGraph agent 多輪呼叫 search 為常態。一個複雜查詢可能觸發 3 次 search，累積 15 個 chunks，同一個 document_id 出現多次屬預期行為。本系統於應用層以 dict 進行去重，每份文件保留最高 relevance 之 chunk。

去重操作未放於 ChromaDB 層之原因為：ChromaDB 之 `where` 子句不支援 SQL `GROUP BY` 語意。

機制 4：拒答情境之 sources 抑制

典型失效情境如下：viewer 角色查詢 SOP，pre-filter 排除所有 SOP chunks，fallback 結果包含「具存取權但與主題無關」之公用文件；LLM 回應「無存取權限」，但 sources 面板仍列出 5 筆公告文件。此情境造成使用者認知衝突。

本系統於串流結束後掃描 LLM 回應，若內容包含 `["沒有權限", "沒有工具", "無法查詢", "找不到相關"]` 等關鍵字則清空 sources。

未採用 LLM 進行拒答判定之原因為：(1) 額外 LLM 呼叫之延遲為 500 ms-2 s；(2) 成本上升約 20%；(3) 引入新的失敗模式。字串包含判定可涵蓋約 90% 之場景，且具備零延遲、零成本、零依賴之特性。

6.3 Sources 之下游動作整合

Sources 不僅為來源顯示，亦支援點擊下載之下游操作。實作上之關鍵設計：

- `sources` schema 額外攜帶 `current_version_id` 欄位，於 pipeline 步驟 ⑤ 透過 batch query 自 DB enrich（不採 ChromaDB metadata，因 `current_version_id` 為動態值）。
- 前端 blob download 複用既有 `can_access_document` 守護之 download endpoint，不引入新的對外 surface 與對應之 RBAC 檢查點。
- 實作過程中識別出一項前端 bug：本專案之 axios response interceptor 已預先執行 `return response.data`，初版前端寫法 `new Blob([res.data])` 因此下載出字面 `"undefined"`。修補方式為直接使用 `res` 作為 blob 來源。

7. 輔助模組

7.1 認證模組

7.1.1 JWT Auto-refresh

JWT 認證之主要設計決策列於 Table 7.1。

Table 7.1 JWT 認證之設計決策與取舍。

設計項目	取舍依據
Access token 30 分鐘 + Refresh token 7 天	洩漏影響範圍與使用體驗之平衡
Refresh token 儲存於 DB 且可撤銷	純 JWT 無撤銷機制；引入半 stateful 設計以換回撤銷能力
DB 僅儲存 SHA-256 hash	Refresh token 為 32 bytes 之機器生成高熵字串，破解空間為 2^{256} ，無需 bcrypt 等 slow hash（bcrypt 設計用於人類密碼之低熵情境）
Axios response interceptor 攔截 401 自動 refresh	被動 refresh 將 expiry 判斷之權威交由 server，避免依賴 client 時間
Race condition 防護 (subscriber queue)	多請求同時 401 時僅一個觸發 refresh，其餘排隊取得新 token，避免 thundering herd

需釐清之概念：Stateless 之意義為「不於 server 端儲存 session state 以避免 multi-instance 共享需求」，並非「不查詢資料庫」。RBAC 等業務邏輯仍需查詢 user 表。

7.1.2 帳號生命週期

- Admin 建立帳號時不設定密碼，發送啟用信由使用者自行設定，避免明文密碼經 admin 之路徑。
- 忘記密碼 endpoint 不論 email 是否存在皆返回相同訊息，避免 email enumeration 攻擊。
- 啟用信與重設密碼之流程約 95% 相同，但拆分為兩個 endpoint。原因為兩者語意不同（首次設定 vs. 找回密碼），分開後 audit log 與 rate limit 配置可獨立調整。

7.1.3 登入安全

- 連續登入失敗 5 次觸發帳號鎖定 15 分鐘。
- 雙維度 rate limit：帳號級防鎖死攻擊、IP 級防分散式撞庫。
- 所有登入嘗試寫入 LoginHistory 表，目的非僅為 audit log，而為支援未來之「異常登入模式」查詢需求。

本系統明確選擇不實作完整版之 session 管理（強制登出所有 session、新登入踢出舊 session、各裝置 session 清單）。原因為完整實作需配合 device fingerprinting 與 suspicious activity detection 之基礎建設，工作量超出示範性系統之範圍。部分實作將造成使用者對安全能力之錯誤預期，故選擇全部不做。

7.2 文件管理模組

7.2.1 組織 CRUD

組織管理子系統之主要設計如 Table 7.2。

Table 7.2 組織管理子系統之主要設計。

設計項目	依據
Role 與 User 之間採 junction 表並含 <code>scope_type</code> 欄位	同一個 role 可全公司指派或限定部門指派，支援未來多租戶擴充
四個角色 (admin / manager / employee / viewer)	Viewer 不具 <code>document:create</code> ，employee 不具 <code>document:delete</code> ；manager 持 <code>user:view</code> 因此通過 <code>require_admin</code> ，但仍受 <code>can_access_document</code> 過濾
後台 admin 與資料面 admin 採分離之判斷標準	後台管理 <code>require_admin</code> 以 <code>startswith("user:")</code> 為準；資料面 <code>can_access_document</code> 以 <code>ai:admin_tools</code> 為準，支援「能審核但不能跨部門看資料」之中階主管角色

7.2.2 資料夾、文件、版本與收藏

樹狀資料夾與扁平結構並存：folder 為可選欄位，文件可隸屬於 folder 或獨立存在。樹狀結構透過自我參照 (`parent_id`) 實作；folder 刪除於路由層檢查 `has_children` 與 `has_documents`，schema 層之 `parent_id ondelete=RESTRICT` 為對應之後端防護，構成 defense in depth。

版本控制採新建不覆蓋：新版本上傳為 INSERT 操作而非 UPDATE，`current_version_id` 指標待審核通過後始切換。被駁回之版本不會影響 `current_version_id`，下載操作仍指向上一個 approved 版本。

檔案系統採 `{年/月}/{uuid}` 雙層分組：避免單一目錄之檔案數量無上限增長。

SHA-256 checksum 用於擋重複上傳：相同檔案再次上傳返回 400 Bad Request。

軟刪與硬刪採分離語意：DB 層採軟刪以滿足 audit 需求，ChromaDB 採硬刪以保證 AI 搜尋之即時一致性。兩個儲存系統之刪除語意刻意不同。

7.2.3 文件審核流程

狀態機採單向 enum：`pending → approved` 或 `pending → rejected`，不存在第三種狀態。駁回操作必須提供原因。

關鍵設計：`approved` 狀態為 ChromaDB 寫入之唯一觸發點。被駁回之文件不會進入向量資料庫，故無需依賴 metadata 過濾於搜尋時排除 pending/rejected 文件。

對比反方案「pending 文件亦寫入 ChromaDB，於搜尋時以 metadata 過濾 `status=approved`」：

- 反方案依賴 `where` 子句之 `status` 過濾，存在 `metadata` 遺漏或 `filter` 遺漏之失敗路徑。
- 本設計於物理層保證未審核文件不進入向量資料庫，屬 `fail-safe` 性質。

兩個獨立狀態機：

- `status` (pending / approved / rejected) — 反映人工審核狀態。
- `vectorization_status` (pending / processing / done / failed) — 反映機器處理狀態。

Approved 後 若 `vectorization` 失敗，最終狀態為 `status=approved + vec_status=failed`。
審核決定不因機器處理失敗而 `rollback`，使「人類審核責任」與「機器處理責任」於模型層分離。

8. 部署與運維

8.1 Stateless 設計與冷啟動

EMKS 部署於 Render（免費方案於 15 分鐘 idle 後休眠）與 Vercel。Render 重啟時 SQLite、ChromaDB、上傳檔案均消失，stateless 為本系統之設計特性而非限制。

Seed 流程採 lifespan 背景非同步執行：

- 不於 entrypoint 同步執行之原因：Render health check 於 seed 完成前可能判定容器啟動失敗並重啟，造成永久無法進入服務狀態。
- 實作上採 `asyncio.create_task(_run_seed_background())` 並儲存於 `app.state.seed_task` 以防 GC。
- `/health/ready` 透過查詢 admin user 是否存在判定可登入狀態。

需指出之設計細節：「ready 等同可登入」不等於「全功能可用」。`_seed_rbac` commit 後 admin 已存在（ready=true），但 `_seed_documents` 可能仍在執行；使用者登入後問 AI 可能搜尋不到部分文件。此屬示範性系統可接受之取捨。

前端冷啟動 overlay：輪詢 `/health/ready`，加 500 ms 延遲避免閃爍；`timeout` 設為與 `interval` 相同之 3 秒，避免請求堆積與 axios 全域 loading overlay 衝突。

8.2 多環境配置

本系統採 12-Factor 配置原則：所有可變配置經由環境變數注入，本地 docker-compose 與雲端部署共用同一份 code、差異於 env。

三個關鍵環境變數之失誤行為設計如 Table 8.1。

Table 8.1 關鍵環境變數之失誤行為。

環境變數	未設定之行為
CORS_ORIGINS	<code>"".split(",") = [""]</code> ，所有 origin 不 match，請求全擋。觀察上屬 fail-loud。
JWT_SECRET_KEY	採用 placeholder 預設值，屬 silent fail，為最危險之失誤模式。雲端部署採 Render 生成之真實 secret。
DATABASE_URL	Lazy fail。create_engine 不立即連線，health check 不查 DB 故持續返回 200，但業務 endpoint 全 500。

ChromaDB 採雙模式設計：本地透過 `HttpClient` 連接 docker volume，雲端透過 `PersistentClient` 寫入 ephemeral disk。Ephemeral 儲存為 stateless 特性之物理基礎。

Email 模組同採雙模式設計：`EMAIL_MODE=console` 將信件內容輸出至 log（適用於開發與 demo 環境）；`EMAIL_MODE=smtp` 連線實際 SMTP server（Gmail 或他者）。雲端因 Render 免費方案擋 outbound SMTP 而於部署環境退化為 console 模式，詳見 §10.4。

Dockerfile shell form CMD 之副作用：`CMD uvicorn app.main:app --host 0.0.0.0` 之 shell form 使 sh 成為 PID 1，SIGTERM 訊號不直達 uvicorn，重啟流程經 10 秒 grace timeout 強制終止。此項列為已知技術債。

9. 工程品質

9.1 Defense in depth 之具體場景

Defense in depth 之意義不僅止於「多一層防 prompt injection」，亦包含「多一層防自身未來之 regression」。本系統之具體應用如 Table 9.1。

Table 9.1 Defense in depth 設計於本系統之具體場景。

場景	對應防線
RBAC	三層機制：bind_tools 結構過濾、tool 內驗證、ChromaDB pre-filter
Resource access	resource-level（存取資料權）+ role-level（操作執行權）兩條正交軸
Folder delete	路由層檢查 <code>has_children + has_documents</code> ，schema 層 <code>ondelete=RESTRICT</code>
審核流程	approve / reject 路由各自驗證一次 <code>is_deleted</code> （fetchPending 已過濾，路由層再驗以避免多 admin 並發場景之 race）

9.2 Fail-safe 與 Fail-loud 之設計分類

系統設計之失誤行為依屬性分類如 Table 9.2。

Table 9.2 失誤行為之屬性分類。

設計	屬性
Approved 為 ChromaDB 寫入唯一觸發點	Fail-safe — 未審核文件不存在於向量資料庫之物理保證
CORS_ORIGINS 未設定請求全擋	Fail-loud — 配置錯誤立即可見
JWT_SECRET_KEY placeholder	Fail-silent — 最危險之失誤模式，雲端強制設定真實 secret
<code>/health/ready</code> 內含 try/except	Fail-safe — DB 未就緒時靜默返回 ready=false 而非 500

9.3 Idempotency

- 向量化 pipeline：每次寫入前刪除 chunk_id 前綴匹配之所有條目，重複執行 N 次最終狀態相同。
- Seed 流程：lifespan 啟動時 `_already_seeded` 檢查 admin 是否存在，存在則 skip。
- 已知缺陷：冪等粒度不足。Partial seed 中斷後（如 `_seed_one_document` 於 ORM 步驟 raise），DB 留下部分文件；下次冷啟動因 admin 存在而 skip 整段，剩餘文件無法補上。

9.4 測試覆蓋

本系統之測試範圍如 Table 9.3。

Table 9.3 測試覆蓋範圍。

範圍	內容
Auth	5 個 test (login / token / refresh 等)
RBAC	5 個 test (4 個角色 × 主要 endpoint 之矩陣抽樣)
Agent eval	1 條 parametrize 4 case 含 spy 工具，於開發過程識別並修補 prompt 規則 #3
整合測試	雲端與本地 smoke test 通過

已知測試覆蓋缺口：「7 個 document endpoint × 4 種角色」之完整矩陣未系統化執行。雲端與本地 smoke test 通過，但補上對應之單元測試對 regression 防護更為充分。

9.5 可觀測性

- **結構化 logging**：採用 loguru 配合 `request_id` ContextVar，每筆 log 攜帶 trace ID。
- **Rate limit**：採用 slowapi，`/ai/agent` 限制為 5/分鐘 + 30/天 per user，`/auth/login` 限制為 10/分鐘 per IP，超限返回 429。
- **缺口**：未實作 metric (Prometheus、DataDog) 與 alert，於示範性系統之範圍內未列為需求。

10. 已知限制與技術債

本章依嚴重性排序列出系統之已知限制。

10.1 AI 品質保證

AI 品質相關之技術債如 Table 10.1。

Table 10.1 AI 品質保證之技術債。

項目	描述	影響	補救路徑
無 RAG eval 資料集	Threshold 0.35 為 demo 場景之經驗值，缺乏 precision@k / recall@k 等系統化指標	規模擴大後之 regression 偵測能力不足	建立 eval dataset、跑 baseline、改參數時執行 A/B 測試
無「該不該 call KB」eval	偶發之 agent 不 call KB 情境僅能透過使用者回報或人工檢驗識別	D4 hard layer 之失靈無法量化偵測	同上，附加 expected_tool 標註
System prompt 迭代未版本化	Prompt 修改無 A/B test 與 rollback 機制	Prompt 修改造成之退化無法量化偵測	Prompt 進入 config 管理，修改寫入 commit message，配合 eval 執行

10.2 架構技術債

架構層級之技術債如 Table 10.2。

Table 10.2 架構層級之技術債。

項目	描述	保留原因
延用 sync <code>stream()</code>	LangGraph 提供 async-native <code>astream()</code> ，換用後可消除 <code>iterate_in_threadpool</code> 與三層防護中之兩層硬性約束	Sync 路徑經三輪修補後已穩定，換用 <code>astream</code> 需重構整段 generator 與 <code>ContextVar</code> 處理，於示範性系統之範圍內 cost 高於 benefit
Permission JOIN 三表無 cache	每個請求查詢 User → UserRole → Role → RolePermission → Permission	規模小無感；規模大時可加 Redis TTL cache (10-60 秒)
Provider abstraction 未實作	OpenAI 直接耦合，無 <code>LLMClient</code> interface	切換 provider 之動作量大，列於 backlog
<code>iterate_in_threadpool</code> 不保證同一 worker thread	Starlette 內部實作可能跨 thread 切換	Demo 階段未觸發；徹底修補方式為換用 <code>astream</code>
Dockerfile shell form CMD 副作用	sh 成為 PID 1，SIGTERM 不直達 uvicorn	重啟流程經 10 秒 grace 強制終止；docker-compose 改為 exec form 即可修補

10.3 RBAC 與 Audit 缺口

RBAC 與 audit 層級之缺口如 Table 10.3。

Table 10.3 RBAC 與 audit 層級之缺口。

項目	描述
完整版 session 管理未實作	強制登出所有 session、新登入踢舊 session、各裝置 session 清單； 本系統採全有或全無策略，未實作部分版本
Approve 並發無 row lock	多 admin 同時 approve 將導致雙倍 OpenAI 成本與 reviewed_by 互相覆蓋
Vectorization 失敗無重試入口	approved + vec_failed 狀態下，再次呼叫 approve 被 status check 攔下； 唯一 workaround 為使用者重新上傳
current_version_id 無 monotonic guard	多個 pending 版本按非時間順序 approve 將使 current 切回較舊版本
fetchPending 採拉模型	新 pending 不會推送至已開啟 review tab 之 admin
Favorite endpoint 缺 can_access_document	員工可 POST 收藏自身不可見之部門文件；GET favorites 暴露 filename 等 metadata（下載操作仍 fail-secure）
Folder tree GET 缺 RBAC 過濾	員工可見全部部門之 folder 名稱，構成 metadata leak
Folder update 缺 cycle detection	多層循環將使 folder 自 build_tree 流程失蹤（單層自我參照已擋）

10.4 部署與運維

部署運維層級之限制如 Table 10.4。

Table 10.4 部署運維層級之限制。

項目	描述
冷啟動 2 分鐘 seed	Render 免費方案之限制；付費方案可保留 instance 不休眠
冪等粒度不足	Partial seed 中斷後無恢復機制；「全部完成」或「全部未開始」之間之中間狀態無法表達
反向代理 buffer 風險	已加 X-Accel-Buffering: no header 緩解；無法 100% 保證所有反向代理皆遵循此 header
雲端 SMTP outbound 受阻	Render 免費方案擋 outbound port 25/465/587（防 spam relay 政策，幾乎所有 cloud PaaS 皆同）。本地 docker-compose 透過 Gmail SMTP 寄信正常； 雲端啟用信與重設密碼信寄不出，於部署環境退化為 EMAIL_MODE=console。 補救路徑為換用 HTTPS API 之 email 服務（Resend / SendGrid / AWS SES 等）， 於示範性系統範圍內未實作

11. 討論

11.1 Persona 假設之適用範圍

本系統之設計決策 pin 於以下 persona：80 人規模、台灣 B2B SaaS、內部 SOP 文件為主、員工每天約 50 次 AI 查詢。各項決策（ChromaDB embedded、top_k=5、threshold 0.35、SSE 而非 WebSocket、單一對話框）皆於此 persona 下達成局部最佳。換用其他 persona 後之翻盤方向如 Table 11.1。

Table 11.1 不同 persona 下之決策翻盤方向。

Persona	翻盤方向
千萬級文件規模	ChromaDB embedded → Pinecone managed 或 pgvector
高風險領域（法律、醫療）	top_k 提升至 15-20 + re-ranking layer + LLM judge
多人協作場景	SSE → WebSocket（雙向通道之必要性）
純工程使用者（DevOps）	統一入口反而干擾，可能需要分 tab
多模態應用	Tool 集大幅擴充，含 OCR、Whisper、vision embedding

設計決策具備適用範圍乃一般性特徵，本系統明確 pin 至特定 persona 以避免泛化主張。

11.2 與生產系統之差距

本系統與生產級實作之差距可分為「可直接移植」與「需顯著加工」兩類。

可直接移植之部分：

- RBAC 模型（4 角色、permission code 設計、scope 欄位）
- 三層權限驗證之設計思路
- 統一入口架構決策
- SSE 三層防護機制
- 文件審核流程之狀態機設計

需顯著加工之部分：

- SQLite + ChromaDB embedded 替換為 managed services（RDS + Pinecone / pgvector）
- Multi-tenancy 完整支援（row-level security 或 schema-per-tenant）
- Metric、trace、alert（OpenTelemetry + Prometheus / DataDog）
- RAG eval pipeline 與 A/B test 基礎建設
- Token usage 與 cost tracking
- Audit log 自 LoginHistory 擴展至全 mutation 追蹤
- Background job queue（目前 vectorization 採 lifespan task 之臨時方案）

11.3 未來工作

於示範性系統範圍結束後，可繼續推進之具體方向：

1. **RAG eval pipeline**：建立 golden dataset（手寫 50-100 query 與 expected docs），跑 precision@k / recall@k baseline，執行 threshold、chunk size、embedding model 之 A/B 測試。
2. **Provider abstraction**：定義 `LLMClient` interface 並實作 OpenAI 與 Claude 兩個版本，驗證跨 provider 之 prompt 可移植性。
3. **Multi-tenant 完整驗證**：評估 row-level security 與 schema-per-tenant 之取捨，驗證現有 scope 欄位設計之 scale 能力。
4. **Astream 重構**：評估換用 LangGraph async-native 後三層防護之結構變化，特別是 ContextVar 跨 thread 議題之消除可能性。

12. 結論

EMKS 為一示範性系統，其目的非生產級部署，而為展示 LLM 整合至帶業務約束系統之端到端工程實作樣本。

三項核心決策具備可被後續系統參考之價值：

1. **RBAC 三層權限驗證機制 + pre-filter 屬性釐清**：權限規則應於每條對外 surface 各自實作；resource-level 與 role-level 為兩條正交軸，完整 RBAC 需於兩者皆建立驗證。
2. **統一入口優於分流介面**：Agent 形態之系統應由 Agent 自身完成 dispatch；產品複雜度應由前端向後端轉移，使介面複雜度對使用者透明。
3. **SSE 三層防護機制之結構性意義**：跨 async/sync 邊界之串流場景具有可預期之三類故障點；分層防護之 narrative 多於迭代過程中顯現，描述時應區分「結構之適用性」與「結構之來源」。

本系統之設計過程驗證以下觀察：面對 AI 整合所帶來之新威脅模型（prompt injection、tool call 誤判、幻覺），單一防線之設計無法充分對應。Defense in depth 不僅為 security 領域之概念詞彙，亦為 AI 應用層工程之必要設計範式。

13. 附錄

A. 技術選型總表

系統採用之主要技術及其替代方案分析如 Table A.1。

Table A.1 技術選型總表。

層級	選用	主要替代	選擇依據
後端框架	FastAPI	Django REST、Flask	Async-ready、Pydantic 整合、Swagger 自動生成
ORM	SQLAlchemy 2	SQLModel、Tortoise	成熟度、junction 表支援
關聯式 DB	MySQL 8	PostgreSQL	目標客戶環境慣例（於 demo 不具決定性）
向量 DB	ChromaDB embedded	Pinecone、pgvector、Weaviate	零運維、Python 整合、demo 規模匹配
LLM 框架	LangGraph	LlamaIndex Agent、自寫實作	State machine、multi-step、tool calling 之成熟度
Condense	LlamaIndex condense_question	自寫 prompt	內建 prompt 經大量使用驗證
Embedding	OpenAI text-embedding-3-small (1536d)	text-embedding-3-large (3072d)	Small 於 SOP 場景之 accuracy 邊際提升 +2.3%，成本提升 6.5x，不符 demo 場景效益
Chat LLM	OpenAI gpt-4o-mini	gpt-4o、Claude、Llama	Tool calling 訓練成熟度與成本之取捨
前端框架	Vue 3 + Vite	React	開發者既有經驗
狀態管理	Pinia	Vuex	Vue 3 官方推薦
HTTP client	axios	fetch	Interceptor 機制利於 401 auto-refresh 實作
後端部署	Render	Railway、Fly.io	免費方案 + GitHub 整合
前端部署	Vercel	Netlify、Cloudflare Pages	SPA 處理、環境變數 build-time 注入

B. 資料模型概要

主要資料模型如 Listing B.1（共 21 張表，列出關鍵 12 張）。

Listing B.1 主要 entity 結構 (簡化版) 。

```
User
├ id, email, hashed_password, department_id, is_active
└ relationships: UserRole, UserToken, Favorite, LoginHistory

Department
└ id, code, name

Role
└ id, name, permissions: RolePermission

Permission
└ id, code (e.g. "user:create"), name

UserRole (junction)
├ user_id, role_id
└ scope_type (global / department), scope_id

RolePermission (junction)
└ role_id, permission_id

Document
├ id, title, permission_level, department_id, folder_id
├ current_version_id, is_deleted
└ relationships: DocumentVersion, Favorite

DocumentVersion
├ id, document_id, version_number, file_path, checksum
├ status (pending/approved/rejected), reject_reason
├ vectorization_status (pending/processing/done/failed)
└ uploaded_by, reviewed_by

Folder
├ id, name, parent_id (self-reference, ondelete=RESTRICT)
└ relationships: Document

ChatHistory
└ id, user_id, conversation_id, role, content, sources_json

UserToken (refresh token)
├ id, user_id, token_hash (SHA-256)
└ expires_at, is_revoked

LoginHistory
└ id, user_id (nullable), ip, success, attempted_at
```

ChromaDB collection 結構如 Listing B.2 。

Listing B.2 ChromaDB 之 collection 結構。

```
collection "knowledge_base"  
  documents: chunk text  
  embeddings: 1536-dim (text-embedding-3-small)  
  metadatas: {  
    document_id, document_title, chunk_index,  
    permission_level, department_id  
  }  
  ids: "doc{doc_id}_chunk{i}"
```

C. 主要 API endpoint

系統之主要 API endpoint 列於 Listing C.1。

Listing C.1 主要 API endpoint。

```
Auth
  POST    /auth/login
  POST    /auth/refresh
  POST    /auth/logout
  POST    /auth/forgot-password
  POST    /auth/reset-password
  POST    /auth/activate-account

AI
  POST    /ai/agent                (SSE, 唯一入口)

Knowledge Base
  GET     /knowledge/documents
  POST    /knowledge/documents      (upload)
  GET     /knowledge/documents/{id}
  DELETE  /knowledge/documents/{id}
  POST    /knowledge/documents/{id}/versions      (新版本)
  GET     /knowledge/documents/{id}/versions/{vid}/download
  GET     /knowledge/folders/tree
  POST    /knowledge/folders
  POST    /knowledge/favorites/{doc_id}
  DELETE  /knowledge/favorites/{doc_id}

Knowledge Review (admin only)
  GET     /knowledge/review                (pending 列表)
  POST    /knowledge/review/{vid}/approve
  POST    /knowledge/review/{vid}/reject

Organization (admin only)
  CRUD    /users
  CRUD    /departments
  CRUD    /roles
  CRUD    /permissions

Health
  GET     /                            (liveness)
  GET     /health/ready                 (readiness - 查詢 admin 是否存在)
```

D. 參考文獻

Defense in depth 與 Security Pattern

- OWASP API Security Top 10 (2023). 其中 BOLA (Broken Object Level Authorization) 與 BFLA (Broken Function Level Authorization) 分別對應本文 §3.5 所述之 resource-level 與 role-level 驗證缺陷類型。
- Saltzer, J. H., & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278-1308.

RAG

- Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
- LlamaIndex Documentation. `condense_question` prompt template 之設計。

LangGraph

- LangChain Documentation. StateGraph、ToolNode、bind_tools 之 API 規格。
- LangGraph multi-agent pattern 文件。

LLM Tool Calling

- OpenAI Function Calling Documentation. GPT-4o tool calling bias 之行為說明。
- Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*.

SSE 與 Streaming

- HTML Living Standard. Server-Sent Events 規範。
- Starlette Documentation. `StreamingResponse` 與 `iterate_in_threadpool` 之實作。

Python ContextVar

- PEP 567 — Context Variables.
- Python 官方文件：`contextvars.copy_context()` 於 threadpool 之語意。

本白皮書為 EMKS 技術示範性系統之綜述文件。實作細節之深度討論、決策歷程記錄、以及對應之內部技術文獻分別存於各專案文件中。